

AiSD Lista 1

Dominik Kowalczyk

7 listopada 2024

1 Algorytm sortowania INSERTION SORT (przez wstawianie)

W podstawowej wersji (bez modyfikacji) algorytmu sortowania przez wstawienie wyróżniamy następujący fragment kodu:

Listing 1: Kod dla funkcji sortującej przez wstawienie

```
1 void INSERTION_SORT(int A[], int n) {  
2     for (int i = 1; i < n; i++) {  
3         int key = A[i];  
4         int j = i - 1;  
5         while (j >= 0 && A[j] > key) {  
6             A[j + 1] = A[j];  
7             j--;  
8         }  
9         A[j + 1] = key;  
10    }  
11 }
```

Powyższy fragment kodu zaczyna się od funkcji `for`, która przechodzi kolejno po każdym elemencie tablicy A . Naszą iterację zaczynamy od drugiego elementu tablicy (z indeksem 1) (zakładając, że 0 jest już posortowany). Zmienna *key* umożliwia przechowanie wartości wstawianego elementu zapobiegając nadpisaniu oraz pozwalając na porównywanie tej wartości z kolejnymi elementami posortowanego ciągu. Gdy napotkany element w posortowanej części jest większy od wstawianego, przesuwamy go o jedną pozycję w prawo, aby zrobić miejsce na wstawienie nowego elementu. Jeśli element w posortowanej części jest mniejszy lub równy wstawianemu, wówczas zapisujemy wstawiany element na kolejnej pozycji.

Modyfikacja INSERTION SORTA polega na wstawianiu "na raz" dwóch kolejnych elementów tablicy. Zatem duża część kodu działa w dość analogiczny sposób.

Listing 2: Fragment 1 funkcji sortującej przez wstawienie z modyfikacją

```
1 void INSERTION_SORT_MOD(int A[], int n) {  
2     for (int s = 1; s < n - 1; s += 2) {  
3         int pierwszy = A[s];  
4         int drugi = A[s + 1];  
5         if (pierwszy > drugi) {  
6             swap(pierwszy, drugi);  
7         }  
8         int k = s - 1;  
9         while (k >= 0 && A[k] > pierwszy) {  
10             A[k + 1] = A[k];  
11             k--;  
12         }  
13         A[k + 1] = pierwszy;  
14         A[k + 2] = drugi;  
15     }  
16 }
```

Pętla zaczyna się od indeksu 1 i iteruje z krokiem +2, co pozwala jednocześnie rozpatrywać dwie wartości – *pierwszy* i *drugi*. Następnie porównujemy je za pomocą funkcji if i jeśli *pierwszy* jest większy zamieniamy miejscami. Funkcja while działa już na tej samej zasadzie co w kodzie bez modyfikacji.

Listing 3: Fragment 2 funkcji sortującej przez wstawienie z modyfikacją

```

1  if (n % 2 == 0) {
2      int ostatni = A[n - 1];
3      int k = n - 2;
4      while (k >= 0 && A[k] > ostatni) {
5          A[k + 1] = A[k];
6          k--;
7      }
8      A[k + 1] = ostatni;
9  }

```

Przedstawiona modyfikacja jednak wymaga rozważenia przypadku, czy tablica A ma długość parzystą, czy nieparzystą. Problem pojawia się gdy n jest parzyste, gdyż ostatni sortowany element nie ma pary ($ostatni = A[n - 1]$). Dla k przypisujemy przedostatni indeks tablicy, które też będzie miejscem rozpoczęcia wstawiania elementu $ostatni$. Pętla *while* działa identycznie jak ta w podstawowym INSERTION SORT, z uwagą, że naszym *key* jest *ostatni*.

2 Algorytm sortowania MERGE SORT (przez scalenie)

Listing 4: Fragment 1 funkcji sortującej przez scalenie bez modyfikacji

```

1  void MERGE(int A[], int p, int s, int k) {
2      int n1 = s - p + 1;
3      int n2 = k - s;
4      int L[n1 + 1];
5      int P[n2 + 1];
6      L[n1] = INT_MAX;
7      P[n2] = INT_MAX;
8  }

```

W przypadku MERGE SORTA ważną rolę odgrywa funkcja MERGE, na której na dobrą sprawę zbudowany jest MERGE SORT. MERGE przyjmuje cztery argumenty: p - początek, s - środek oraz k - koniec. Za ich pomocą dzielimy tablicę na dwie części: lewą ($n1 = s - p + 1$) oraz prawą ($n2 = k - s$). Definiujemy funkcje pomocnicze L i P o pojemności kolejno $n1 + 1$ oraz $n2 + 1$. Następnie

ustawiamy ostatni element na "napewno większy" od największego elementu tablicy $A[]$.

Listing 5: Fragment 2 funkcji sortującej przez scalenie bez modyfikacji

```
1 int i = 0;
2 int j = 0;
3 for (int l = p; l <= k; l++) {
4     if (L[i] <= P[j]) {
5         A[l] = L[i];
6         i++;
7     } else {
8         A[l] = P[j];
9         j++;
10    }
11 }
```

Po przypisaniu z lewej części do zmiennej L i z prawej części P resetujemy wskaźniki i oraz j (które wskazują naszą pozycję w kolejno w L i P).

Instrukcja $if(L[i] \leq P[j])$ sprawdza, czy bieżący element z L jest mniejszy lub równy bieżącemu elementowi z P .

Jeśli tak, to mniejszy element $L[i]$ jest wstawiany do tablicy $A[l]$, a wskaźnik i jest zwiększany o 1, by przejść do następnego elementu w L .

Jeśli nie, to mniejszy element $P[j]$ jest kopiowany do $A[l]$, a wskaźnik j jest zwiększany o 1, by przejść do następnego elementu w P .

Listing 6: Fragment 3 funkcji sortującej przez scalenie bez modyfikacji

```
1 void MERGESORT(int A[], int p, int k) {
2     if (p < k) {
3         int s = (p + k) / 2;
4         MERGESORT(A, p, s);
5         MERGESORT(A, s + 1, k);
6         MERGE(A, p, s, k);
7     }
8 }
```

Po przeanalizowaniu funkcji `MERGE`, możemy przejść do funkcji `MERGE SORT`, która wylicza srodek tablicy i wywołuje kolejno sortowanie z lewej strony, prawej strony po czym scala obie strony.

Listing 7: Fragment 1 funkcji sortującej przez scalenie z modyfikacją

```

1 void MERGEMODYFIKOWANY(int A[], int p, int s1, int s2, int k) {
2     int n1 = s1 - p + 1;
3     int n2 = s2 - s1;
4     int n3 = k - s2;
5     int L[n1 + 1];
6     int S[n2 + 1];
7     int P[n3 + 1];
8     L[n1] = INT_MAX;
9     S[n2] = INT_MAX;
10    P[n3] = INT_MAX;
11 }

```

Modyfikacja MERGE SORTA polegała na podzieleniu tablicy na trzy części zamiast dwóch, więc cały kod opiera się i ma bardzo podobną budowę do podstawowego MERGE SORTA. Zamiast dwóch części tworzymy trzy:

$$n1 = s1 - p + 1$$

$$n2 = s2 - s1$$

$$n3 = k - s2$$

a jako, że rozdzielamy tablicę. Pozostała reszta MERGA MODYFIKOWANEGO jak i MERGE SORTA MODYFIKOWANEGO przebiega bardzo podobnie.

3 Algorytm sortowania HEAPSORT (przez kopcowanie)

Przy HEAPSORCIE podobnie jak przy MERGESORCIE ciekawe rzeczy dzieją się w funkcjach pomocniczych. Przeanalizujemy zatem HEAPIFY (na kopcach binarnych).

Listing 8: Fragment 2 funkcji sortującej przez kopcowanie bez modyfikacji

```

1 int LEWA(int i) {
2     return 2 * i;
3 }
4 int PRAWA(int i) {
5     return 2 * i + 1;
6 }

```

Kod powyżej definiuje pozycje lewego "dziecka" oraz "prawego" dziecka w kopcu binarnym.

Listing 9: Fragment 1 funkcji sortującej przez kopcowanie bez modyfikacji

```
1 void HEAPIFY(int A[], int i, int n) {
2     int l = LEWA(i);
3     int p = PRAWA(i);
4     int naj = i;
5
6     if (l < n && A[l] > A[i]) {
7         naj = l;
8     }
9
10    if (p < n && A[p] > A[naj]) {
11        naj = p;
12    }
13
14    if (naj != i) {
15        swap(A[i], A[naj]);
16        HEAPIFY(A, naj, n);
17    }
18 }
```

Argumentami jest tablica $A[]$, która reprezentuje nasz "kopiec", i która oznacza indeks od którego zaczynamy "naprawę kopca" oraz n odpowiedzialna za długość tablic.

- $l = \text{LEWA}(i)$
Oblicza indeks lewego dziecka węzła i i przypisuje go do zmiennej l . Używa funkcji LEWA , która zwraca $2 \cdot i$.
- $p = \text{PRAWA}(i)$
Oblicza indeks prawego dziecka węzła i i przypisuje go do zmiennej p . Używa funkcji PRAWA , która zwraca $2 \cdot i + 1$.
- $naj = i$
Ustawia zmienną naj (przechowującą indeks największego elementu) na i . Zakładamy na początku, że węzeł i jest największy wśród siebie i swoich dzieci.
- if ($l < n$ and $A[l] > A[i]$) { $naj = l$; }
Sprawdza, czy lewe dziecko l znajduje się w zakresie kopca

($l < n$), a wartość w tablicy A pod indeksem l jest większa niż wartość w $A[i]$.

Jeśli oba warunki są spełnione, to zmienna naj zostaje ustawiona na l , co oznacza, że największy element jest teraz na pozycji lewego dziecka.

To samo robimy dla p , tylko zmieniamy $A[i]$ na $A[naj]$, gdyż w naturalny sposób przez pierwszego ifa wartość największa mogła się zmienić.

Na sam koniec sprawdzamy, czy znaleźliśmy element większy od naszego początkowego, jeśli tak to zamieniamy miejscami je miejscami. Jeszcze wywołujemy HEAPIFY rekurencyjnie na poddrzewie naj , aby upewnić się, że poddrzewo również spełnia własność kopca.

Listing 10: Fragment 2 funkcji sortującej przez kopcowanie bez modyfikacji

```
1 void BUILD_HEAP(int A[], int n) {
2     for (int i = n / 2 - 1; i >= 0; i--) {
3         HEAPIFY(A, i, n);
4     }
5 }
6
7 void HEAPSORT(int A[], int n) {
8     BUILD_HEAP(A, n);
9
10    for (int i = n - 1; i >= 1; i--) {
11        swap(A[0], A[i]);
12        n--;
13        HEAPIFY(A, 0, n);
14    }
15 }
```

W BUILD HEAPIE inicjujemy pętlę for, która iteruje od środkowego elementu tablicy ($n/2 - 1$) wstecz do pierwszego elementu (indeks 0). Dzięki takiemu iterowaniu nasza funkcja HEAPIFY może poprawnie naprawić kopiec dla każdego poddrzewa, budując kopiec od dołu do góry.

W HEAPSORCIE:

- for ($i = n - 1; i \geq 1; i - -$) {

Inicjuje pętlę, która iteruje od ostatniego elementu tablicy $A[n-1]$ wstecz do drugiego elementu $A[1]$.
przesuwając największy element kopca do jego końcowej pozycji.

- `swap(A[0], A[i]);`
Zamienia miejscami elementy $A[0]$ (korzeń kopca, największy element) oraz $A[i]$ (bieżący koniec kopca), dzięki czemu największy element trafia na właściwe miejsce.

Listing 11: Fragment 1 funkcji sortującej przez kopcowanie z modyfikacją

```
1  int LEWAT(int i) {  
2      return 3 * i;  
3  }  
4  int SRODEK_T(int i) {  
5      return 3 * i + 1;  
6  }  
7  int PRAWAT(int i) {  
8      return 3 * i + 2;  
9  }  
10 void HEAPIFY_T(int A[], int i, int n) {  
11     int l = LEWAT(i);  
12     int s = SRODEK_T(i);  
13     int p = PRAWAT(i);  
14     int naj = i;  
15     if (l < n && A[l] > A[i]) {  
16         naj = l;  
17     }  
18     if (s < n && A[s] > A[naj]) {  
19         naj = s;  
20     }  
21     if (p < n && A[p] > A[naj]) {  
22         naj = p;  
23     }  
24     if (naj != i) {  
25         swap(A[i], A[naj]);  
26         HEAPIFY_T(A, naj, n);  
27     }  
28 }
```


Modyfikacja algorytmu HEAPSORT miała polegać na użyciu kopców binarnych kopce ternarne. Ta modyfikacja najbardziej wpływa na HEAPIFY, gdyż oprócz lewego oraz prawego "dziecka" mamy również środkowe. Tak więc teraz:

funkcja *LEWAT* zwraca $3 * i$

funkcja *SRODEKT* zwraca $3 * i + 1$

funkcja *PRAWAT* zwraca $3 * i + 1$

Więc w zmodyfikowanej wersji HEAPSORTA, w HEAPIFY występuje dodatkowa zmienna *s*, która przy porównywaniu wartości dodaje jedną dodatkową pętlę if.

4 Porównania

4.1 INSERTIONSORT

INSERTION_SORT			INSERTION_SORT_MOD		
ilość elementów	przypisania	porównania	ilość elementów	przypisania	porównania
10	87	39	10	87	31
100	4591	2246	100	4591	2148
1000	501357	250179	1000	501357	249181
10000	50074405	25032203	10000	50074405	25022205
50000	1245241643	622595822	50000	1245241643	622545824

Rysunek 1: Porównanie ilości porównań i przypisań w INSERTION SORT oraz INSERTION SORT MOD.

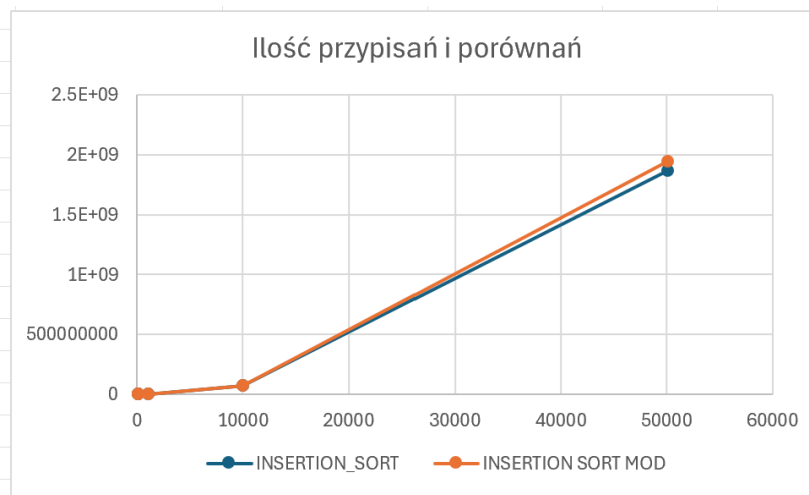
Czas wykonywania sortowania INSERTION SORT: dla 10000 elementów - $95ms$
dla 50000 elementów - $1950ms$

Czas wykonania sortowania INSERTION SORT MOD: dla 10000 elementów - $90ms$
dla 50000 elementów - $2015ms$

Obydwa algorytmy mają bardzo zbliżoną złożoność, która minimalnie jest większa dla zmodyfikowanej wersji insertion sort. Pod względem ilości czasu na sortowanie również wypadają bardzo podobnie.

Złożoność:

- Złożoność optymistyczna: $O(n)$,
- Złożoność pesymistyczna: $O(n^2)$,



Rysunek 2: Wykres ilości porównań i przypisań w INSERTION SORT oraz INSERTION SORT MOD.

4.2 MERGESORT

MERGE_SORT			
ilość elementów	przypisania	porównania	suma
10	168	43	211
100	2591	771	3362
1000	33987	10975	44962
10000	426231	143615	569846
50000	2420871	834463	3255334
	2883848	989967	

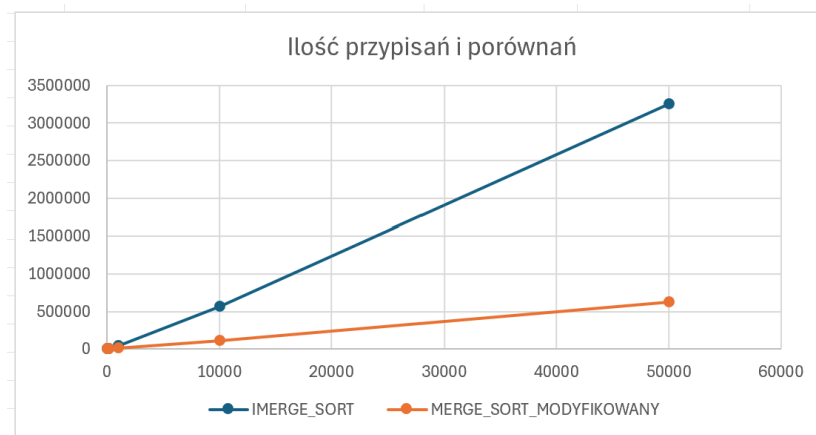
MERGE_SORT.MODYFIKOWANY			
ilość elementów	przypisania	porównania	suma
10	23	27	50
100	203	497	700
1000	2003	7177	9180
10000	20003	93597	113600
50000	100003	529524	629527

Rysunek 3: Porównanie ilości porównań i przypisań w MERGE SORT oraz MERGE SORT MODYFIKOWANY.

Czas wykonywania sortowania MERGE SORT: dla 10000 elementów - $1ms$
dla 50000 elementów - $8ms$

Czas wykonania sortowania MERGE SORT MODYFIKOWANY: dla 10000 elementów - $2ms$
dla 50000 elementów - $8ms$

W tym przypadku zdecydowanie bardziej wydajniejszym i lżejszym algorytmem jest merge z modyfikacją, który dodatkowo nie odstaje wcale bardzo czasowo względem klasycznego merge. Dodatkowo Merge sort jest znacznie szybszy od insertion sortu jak również znacznie lżejszy, przez co i wydajniejszy, co zgadza się z teorią.



Rysunek 4: Wykres ilości porównań i przypisań w MERGE SORT oraz MERGE SORT MODYFIKOWANY.

- Złożoność optymistyczna $O(n \log n)$,
- Złożoność pesymistyczna: $O(n \log n)$,

4.3 HEAPSORT

HEAPSORT			
ilość elementów	przypisania	porównania	suma
10	230	57	287
100	4416	1352	5768
1000	65527	21753	87280
10000	872596	300730	1173326
50000	5120725	1794719	6915444
	6063494	2118611	

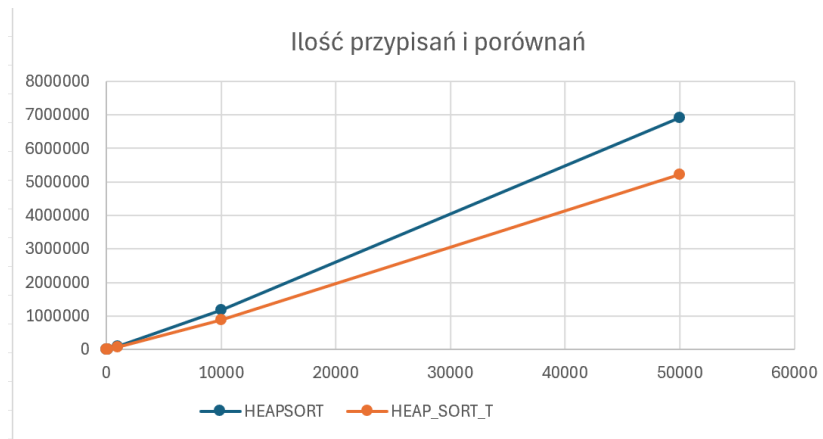
HEAP_SORT_T			
ilość elementów	przypisania	porównania	suma
10	193	38	231
100	3616	961	4577
1000	51631	15001	66632
10000	680540	208905	889505
50000	3970183	1250621	5221004

Rysunek 5: Porównanie ilości porównań i przypisań w HEAPSORT oraz HEAP SORT T.

Czas wykonywania sortowania HEAPSORT: dla 10000 elementów - $2ms$
dla 50000 elementów - $20ms$

Czas wykonania sortowania HEAP SORT T: dla 10000 elementów - $2ms$
dla 50000 elementów - $14ms$

W przypadku algorytmu heapsort widać różnicę złożoności algorytmu po modyfikacji, a przed. Heapsort z kopcami ternarnymi jest wydajniejszy oraz szybszy dla większych zbiorów elementów. Jedna-



Rysunek 6: Wykres ilości porównań i przypisań w HEAPSORT oraz HEAP SORT T.

kowoż w porównaniu do innych, to jest on zarówno szybszy jak i lżejszy pod względem złożoności algorytmu od insertion sort, jednakowoż jest wolniejszy od merge sorta dla dużych zbiorów danych, jak również nie dorównuje dla mergesorta po modyfikacji pod względem złożoności, to ma dosyć zbliżoną względem niemodyfikowanego merge sorta.

- **Złożoność optymistyczna:** $O(n \log n)$,
- **Złożoność pesymistyczna:** $O(n \log n)$,